

Why Katamari?

We wanted to have many ways to qualify to the Online round – and also something to play with for those who can finish the rest of the problem set too quickly and still want to have something to do :) so we came up with this simple but cute TSP-style optimization problem.

Basics

Let's start by determining our final size. For now, let's completely ignore the aspect of minimizing how long it will take us.

Look at the smallest item we haven't "eaten" yet. If we cannot eat it, we obviously cannot eat any other item either, and that means that we are done. And if we can eat it? Obviously, in the optimal solution we will *eventually* eat it, because it's never optimal to stop when there are still objects we can eat. And instead of waiting we can simply eat it first – doing so will grow us, and that won't prevent us from eating other items.

(More formally: Consider any valid order in which we would eat items in the future. Then eating the smallest item and then the other items in their original order is also a valid solution. This is because in this new solution, for each of the other items, we are at least as big as in the original solution when we eat it.)

This gives us a simple greedy algorithm: Sort the items by size and then eat them (smallest to largest) until you either eat them all, or hit an item that's too big to be eaten. Your current size is the maximum possible final size, and the set of objects you have eaten is precisely the set of objects you need to eat when solving the full problem.

(Note: One of the five test cases is chosen so that some of the items are too big to be eaten, to test whether you handle this properly. In the other four tests it is possible to eat all items.)

TSP costs

The "optimal collection times" used by the grader are essentially just the solutions to the Traveling Salesperson Problem (TSP) for that instance – in other words, they correspond to the shortest Hamiltonian path for the set of eventually-edible items. We have computed the exact optimal costs of these paths before the round by using an ILP solver.

A good ILP formulation for Hamiltonian cycles is to start with a boolean variable for each edge and just the constraints that each vertex is incident to two edges. Then, each time you find a solution, verify whether it's a single cycle or multiple ones. If it's multiple cycles, pick one and add a constraint that between those k vertices there can only be at most $k - 1$ edges. Iterate until the shortest viable solution becomes the shortest Hamiltonian cycle.

Easy points

The scoring formula was chosen so that it was possible to score a bit over 50 points very easily. One simple strategy that produces decent results on the test data is to simply always proceed to the closest object we can eat.

One possible next step how to squeeze out more points is also pretty simple: local optimizations. Once you have a valid solution, you can examine its constant-length segments to see whether reordering the actions produces a shorter and still valid solution. This way we can improve our solution somewhat from the greedy baseline to a slightly better local optimum.

Some ideas how to get more points

One issue with the greedy solution (apart from it having some bad TSP cases) is that in the presence of weights, it is sometimes good to travel long distances in the short term in order to gain a lot of weight — this weight could give us the ability to move between items more efficiently. E.g. imagine the following test case:

- We have 20 clusters of “basic” items, each cluster contains 5 layers of items (“cluster” means those items are near each other, items in the same layer have same weight). The weights of the items are such that we can’t collect basic items from layer $N+1$ until we have collected all basic items from all previous layers.
- We have 1 special cluster. This cluster is such that we can visit it right from the start, but the weights in this cluster’s items are such that we can easily gain a lot of weight. E.g. say we start with weight 10, then the items in this cluster could have weights 4, 6, 9, 14, 21, 31, 47, ... — item $(i + 1)$ ’s weight is the largest weight such that we can pick it up with the weight from all the previous items. This grows exponentially, i.e. this cluster allows us to very quickly gain all the weight we need to be able to pick up all items.

If we go greedy, then worst-case we arrive at the special cluster last. This means that we have to traverse the distances between the basic clusters 5 times. On the other hand, if we travel to the special cluster first, then we have to visit the basic clusters only once — a 5x reduction in distance travelled. This is actually what is happening in the last two test cases.

Solution 2: We can somewhat mitigate the above effect as follows. Consider all possible items as the first item to pick up. Then, always pick up the item that has the best weight/distance travelled ratio, until we have accumulated enough weight to be able to pick up all items we could have possibly picked up. Then, do greedy. Yields 67.76 points.

Solution 3: Monte Carlo. In each step, randomly choose the next item to visit (from among currently edible items). This turns out to be quite bad solution (~0 points), but some modifications give something pretty reasonable:

- Instead of assigning equal probability to all candidate items, assign probabilities by distance. More precisely, each item is given value $(\text{-distance}) \times k$ for some k . From these values, we obtain probabilities by applying the softmax function. (Note that this makes k an important parameter.)
- We set k as follows: $k = 10 * \text{curr_best_total_distance} / \text{num_stops}$. That is, k is the average edge distance in the best solution found so far.

This solution yields 71.99 points (but can yield more if we let it run longer).

Solution 4: Finally, we can implement some proper optimizer that takes as input a solution, tries to make some changes and see if we get anything better. We tried something like this:

- Cut the solution into segments, where each segment contains a sufficiently large percentage of items such that they can already be collected at the start of the segment (i.e. are lightweight enough / we are heavy enough).
- Cut each segment randomly into subtours. Then, for each segment (starting with the earliest segment and going forward), try all possible valid orderings of the subtours in that segment and keep the best. (Using memoization, we only have to consider $O(2^s)$ orderings for s subtours.
- Iterate.

After letting it run a bit on the output of our softmax solution, we got the score 77.12.

Solution 5: The Katamari problem can also be reduced to an ILP, but we aren't aware of a formulation that would be as easy for ILP solvers as the TSP formulation. Our success with using an ILP solver for Katamari was mixed — it went well for some test cases, not too well for others. The best version was one that incrementally built the set of items using only those that are or soon become edible, and constructed the solution in parts by running the ILP solver multiple times and always taking a suitable prefix of its solution for the current items and appending it to our solution. This approach also got us over 70 points.

Other approaches can score even more points. If you did something clever, let us know :)

Fun fact: Our optimizer was able to improve the *length* of the best contestant-found solution a little, but not enough to improve its score.